Excepciones Excepcionales

Total, que resulta que el jefe nos ha mandado que diseñemos un interfaz de ventanas para nuestro programa de gestión de patos. Dedicamos unos días a ponerlo todo bonito con botoncitos y tablas y tal, y entonces nos metemos con la parte de conectarlo todo con la Base de Datos, que ya está programado. Y empezamos a marearnos.

Resulta que el tipo que programó la parte del Modelo de la aplicación era muy detallista. Siguió el convenio de Java y los manuales de diseño hasta las últimas consecuencias, de forma que por cada cosa que puede ir mal, hay una excepción diferente. Por ejemplo, así es como hizo el método para comprobar el login & pass contra base de datos.

```
/**
 * Returns true if the user and pass matches DDBB, false otherwise.
 *
 * @param user The user login
 * @param pass The user pass
 * @return True or false
 * @throws UserNotFoundException If the user is not found
 * @throws IncorrectPassException If the pass is not OK
 * @throws InsufficientCredentialsException If the user is not an administrator
 * @throws CancelledeAccountException If the account is cancelled
 */
public boolean checkUserPass(String user, String pass) throws UserNotFoundException,
        IncorrectPassException, InsufficientCredentialsException,
        CancelledeAccountException {

        ...

}
```

Así que ahora nos toca tener en cuenta una media de cuatro excepciones por cada acceso que hagamos a Base de Datos. Y hay unos doscientos métodos diferentes en nuestro programa. Bueno… nos ponemos manos a la obra…

… tan sólo para descubrir después de dedicarle un rato a programar el login que, después de todo, a ti te da igual qué demonios ha ido mal. Ta sólo te interesa si el usuario se puede loguear o no. ¿Cómo lo soluciono? Parece simple. Un try – catch gigante que capture todas las excepciones en tu ventana, y solucionado. Ignoras deliberadamente las Excepciones en cada ocasión. Fácil.

Y sucio. Y chapucero.

Idea loca

Si de verdad lo único que te interesa es saber si una operación ha ido bien o mal, existe una solución común en otros lenguajes de programación, pero relativamente “nueva” en Java. Que no es que sea realmente nueva, pero es que Java tradicionalmente ha utilizado las excepciones para gestionar estas cosas, y no se ha extendido su uso hasta hace relativamente poco.

El **Result Pattern** es una técnica usada para manejar resultados y errores sin excepciones. Lo que hacemos es encapsular el resultado de una operación mediante un objeto que nos dice si todo ha ido bien y dándonos el resultado. Dado que no nos interesa qué ha ido mal, nos oculta lo que ha pasado si se comete algún error.

Dicho esto, aclarar que **Result Pattern** NO es un truco para ignorar las excepciones. En general, es un patrón adecuado cuando:

- Que la operación falle sea algo normal en el flujo de trabajo de nuestro programa. Por ejemplo, un pass incorrecto está contemplado en mi aplicación de patos: simplemente, no le dejamos loguearse.

- Los errores son esperables y no inesperados. Que la base de datos se haya caído NO es un error esperable.

- Nuestro código NO tiene muchos pasos de validación.

- Nuestro código NO tiene un punto central que gestiona todos los errores.

Por tanto, es perfecto para:

- Validaciones de entrada, como el login, en las que no nos interesa qué ha pasado exactamente.

- Accesos a red o a base de datos.

- Procesos encadenados, para evitar que un fallo en un eslabón interrumpa la cadena.

Existen varias formas de implementar esto. Por supuesto, puedes customizarlo como quieras. Pero por ahora, vamos a hacer una prueba para el login de nuestro gestor de patos.

OperationResult

Vamos a diseñar primero el objeto que va a encapsular el resultado de la operación de login. Lo vamos a llamar **OperationResult**. Lo vamos a diseñar de tal manera que sea genérico, y así pueda devolver cualquier tipo de respuesta llegado el caso, y así poder reutilizarlo en otros métodos.

```
public class OperationResult<T> {

    private final boolean success;
    private final T data;
    private final String errorMessage;

    // Constructor for success
    public OperationResult(T data) {
        this.success = true;
        this.data = data;
        this.errorMessage = null;
    }

    // Constructor for failure
    public OperationResult(String errorMessage) {
        this.success = false;
        this.data = null;
        this.errorMessage = errorMessage;
    }

    // Check if the operation was successful
    public boolean isSuccess() {
        return success;
    }

    public T getData() {
        if (!success) {
            throw new IllegalStateException("Error: no data");
        }
        return data;
    }

    public String getErrorMessage() {
        if (success) {
            throw new IllegalStateException("No error message "
                    + "for successful operation.");
        }
        return errorMessage;
    }
}
```

Este código es solamente una implementación simple del patrón. Puedes customizarlo como quieras y necesites.

En OperationResult, podemos cargar cualquier objeto que deseemos como respuesta de una operación (por ejemplo, el login), o añadir un mensaje de texto explicando el error que ha impedido retornar esa respuesta. Vamos a codificar nuestro método getUserPass () pero preparándolo para que devuelva el OperationResult. Este método retorna un objeto User en caso de que encuentre al usuario en Base de Datos.

Accedemos a nuestra clase gestora de BBDD que contiene los métodos de acceso a la BBDD, y hacemos lo siguiente:

1) checkUserPass () es ahora private, para que no pueda ser usado desde fuera de la clase.

2) Añadimos el siguiente método:

```
public OperationResult<Boolean> loginService (String user, String pass) {
    OperationResult<Boolean> ret = null;
    Boolean checked = null;
    try {
        checked = checkUserPass (user, pass);
    } catch (Exception e) {
        return new OperationResult<>(e.getMessage());
    }
    return new OperationResult<>(checked);
}
```

Este método 'oculta' las posibles excepciones generadas por checkUserPass () permitiendo que siempre se retorne un objeto OperationResult, ocurra lo que ocurra. O bien tendrá el resultado de la operación, o bien un mensaje de error.

3) Si quieres ser un poco más limpio, es mejor que todos los métodos de servicio estén en clases aparte. Como idea.

Por tanto… ¿cómo sería nuestra comprobación del login? Simple:

```
public void doCheckUserPassButtonEvent(String user, String pass) {
    OperationResult<Boolean> resultado = loginService(user, pass);
    if (resultado.isSuccess()) {
        System.out.println("La operación retorna: " +
            resultado.getData());
    } else {
        System.out.println("La operación retorna error: " +
            resultado.getErrorMessage());
    }
}
```

Nota Aclaratoria

En este ejemplo, OperationResult retorna un objeto Boolean si todo ha ido bien, lo que obviamente no tiene mucho sentido en realidad. Pero recuerda que el patrón es genérico y flexible. Si tuviésemos que hacer un acceso para retornar todos los patos, no tendríamos que hacer prácticamente nada:

1) Añade un método getAllDucksService () que envuelva a getAllDucks ().

```
public OperationResult<List <Pato>> getAllDucksService()
    OperationResult<List <Pato>> ret = null;
    List <Pato> patos = null;
    try {
        patos = getAllDucks();
    } catch (Exception e) {
        return new OperationResult<>(e.getMessage());
    }
    return new OperationResult<>(patos);
}
```

2) No hace falta tocar OperationResult

3) Añadimos la llamada:

```
public void getAllDucksEvent() {
    OperationResult<List <Pato>> resultado = getAllDucksService();
    if (resultado.isSuccess()) {
        List <Pato> patos = resultado.getData();
        for (Pato pato : patos)
            System.out.println(pato.toString());
    } else {
        System.out.println("La operación retorna error: " +
                resultado.getErrorMessage());
    }
}
```

Ejercicio Propuesto

Crea una Base de Datos de Alumnos. Tenemos que almacenar información en las tablas de Alumno, Nota y Curso. Añade unas clases Gestoras que tengan las operaciones básicas elementales CRUD; y las siguientes operaciones:

- Seleccionar todos los alumnos de un curso y sus notas.
- Seleccionar todos los alumnos de todos los cursos de sobresaliente.
- Seleccionar todos los cursos cuya nota media sea inferior a 5.

A continuación, añade una clase OperationResult y modifica el programa para adatarlo a este patrón.